Домашнее задание 3. Сети.

Формат сдачи:

Все домашние задания по курсу сдаются как страничка в Github-репозитории, который структурирован так, как указано в конце задания ниже. Пожалуйста, обратите внимание на взаимное расположение файлов в репозитории и на их название. Я буду запускать автоматический базовый чекер, который проверяет наличие файлов в репозитории для каждой домашки. Для каждого задания старайтесь хранить листинг команд, которыми вы пользовались – они понадобятся для сдачи. Часто ответы на задание нужно будет писать в README.md в какой-то папке репозитория (воспринимайте это как что-то похожее на домашки по практической биоинформатике).

Уточнение:

Если у вас не Linux, вы можете воспользоваться либо WSL2 (устанавливается автоматически в Windows > 10), либо установить cygwin: **cygwin.org**, позволяющий работать в командной строке. По всем вопросам пишите в чат курса или в личные сообщения преподавателям. При работе с git не забывайте как можно чаще коммитить изменения. В markdown можно вставлять картинки и таблицы – не стесняйтесь это делать, если чувствуете, что это хорошо иллюстрирует ситуацию.

Структура репозитория после третьего ДЗ:

```
.
├── .github
│   └── ...
├── .gitignore
├── hw1
│   └── ...
├── hw2
│   ├── ...
│   └── ...
├── hw3
│   ├── parse_traceroute.py
│   ├── ggtp_client.py
│   ├── ggtp_server.py
│   ├── ggtp_async_server.py
│   ├── DNS_digger.py
│   ├── ARP_scanner.py
│   ├── mac_lookup.py
│   ├── reply_ciphertext.bin
│   ├── reply_signature.bin
│   ├── public.pem
│   └── README.md
├── hw4
│   └── ...
├── hw5
│   └── ...
└── README.md
```

Задание 1. Парсинг traceroute

Разберитесь с API сайта **ipwho.is**. Напишите программу `hw3/parse_traceroute.py`, которая принимает на вход URL (`--url`) и печатает последовательный список троек **(страна, город, IP маршрутизатора)**, по которым проходят пакеты `traceroute` при прослеживании пути до хоста с адресом. Обрабатывайте ситуации, когда на определённом уровне хопа маршрутизатор не отвечает (`* * *`). В вывод программы, файл `url.json` (`url` – параметр при запуске скрипта), выведите JSON-объект: список `"traceroute_steps"`, где каждый элемент – кортеж из трех элементов (страна, город, IP), и количество безответных хопов `"silent_hops"`. Приведите пример и результат запуска какого-нибудь из запросов в `hw3/README.md`. С помощью каких пакетов `traceroute` пробирует сеть?

Задание 2. Guessing game на сокетах.

Построим свой протокол: Guessing Game Transfer Protocol (GGTP) over UDP. Протокол будет реализовывать игру в угадывание числа.

Игра инициируется клиентом при отправке протокольного сообщения `"HELO"` на сервер. В ответ на `"HELO"` сервер проверяет, есть ли IP клиента в таблице активных игра. Если IP клиента в таблице отсутствует, сервер инициализирует игру с клиентом, посчитав CRC32(IP) и инициализировав это как seed для генерации случайного числа в промежутке от 1 до некоторого случайного числа (считайте, что сидом вы устанавливаете генерирацию и верхней границы промежутка из диапазона от 100 до 1000, и числа из него). Затем сервер высылает в ответ клиенту протокольное сообщение `"WLCM lower_bound upper_bound"`, где `lower_bound` – нижняя граница промежутка, `upper_bound` – верхняя.

Клиент, получив `WLCM`-сообщение, и сервер, отправив его, входят в состояние игры друг с другом: клиент отправляет протокольные сообщения `"GUES"` с синтаксисом `"GUES number"`, где `number` – число, которое вводится пользователем с `input`'а. В ответ на `GUES`-сообщение сервер должен возвратить одно из сообщений: `"MORE"`, `"LESS"` или `"BING"`. В случае с `MORE`- и `LESS`-сообщениями сервер не отправляет дополнительной информации (это сообщение служебное и служит для передачи информации о том, больше или меньше загаданное число относительно предположения клиента), в случае с `BING` число угадано, и сервер возвращает победный ключ (64-значная hex-строка, хэш от IP-адреса и загаданного числа вместе). Если число не было угадано клиентом за $\log_2(range)$ попыток, сервер отсылает сообщение `"FAIL"` и удаляет из таблицы записей IP клиента.

Наш протокол работает поверх UDP. Это означает, что датаграммы далеко не всегда будут доходить целыми и невредимыми, или вообще доходить. Реализуйте обработку отсутствия пакетов при том или ином тайм-ауте. Считайте, что тайм-аут может передаваться клиентом в случае с `HELO`-сообщением (например, `HELO 5.0` — значит, клиент ждёт ответ 5 секунд).

Тайм-ауты нужны клиенту и серверу чтобы понимать, когда повторно отсылать датаграммы друг другу. Кроме того, вам нужно обработать случаи, когда сервер или клиент получают UDP-пакеты вне нужного порядка (подумайте, какие это случаи?).

При реализации такой игры мы упустили одну важную вещь – сервер может одновременно играть с несколькими клиентами. Если сервер будет обрабатывать пакеты синхронно от каждого клиента и ждать их, тайм-ауты других клиентов будут выключаться, что приведёт к флуду пакетами с обеих сторон. Прочитайте документацию библиотеки asyncio ещё раз и на основе этой библиотеке реализуйте асинхронный сетевой сервер, поддерживающий протокол GGTP. Ниже представлен пример взаимодействия клиента и сервера по GGTP. Итоговый синтаксис протокольных сообщений реализуемого протокола GGTP:

- HELO [timeout]?;
- WLCM a b;
- GUES N;
- MORE / LESS / BING <key> / FAIL.

В репозитории положите файлы `hw3/ggtp_client.py`, `hw3/ggtp_server.py` и `hw3/ggtp_async_server.py`. В них должны лежать скрипты для клиентской части приложения, и серверной в двух примерах (синхронная и асинхронная). В случае с клиентской и синхронной серверной частью полезно использовать библиотеку `socket`, в случае с асинхронной реализацией сервера за UDP-соединение может отвечать `asyncio`. Можете описать свои ощущения от этого задания в `hw3/README.md`. Тестируйте у себя на `localhost`. Пример взаимодействия между сервером и клиентом по протоколу GGTP:

```
C: HELO 50.0
S: WLCM 1 512
C: GUES 256
S: MORE
C: GUES 400
S: LESS
C: GUES 347
S: BING a17b2c23d4eaa5f678bcb90
```

Задание 3. Анализ DNS записей

Ознакомьтесь с утилитой `dig`. Напишите программу `hw3/DNS_digger.py`, которая запускает поиск всех записей домена, переданного в качестве аргумента `--domain`. Программа запрашивает все записи типов `A` (IPv4 адреса), `AAAA` (IPv6 адреса), `MX` (почтовые сервера), `NS` (имена серверов), `CNAME` (каноническое имя) с помощью `dig`, а также рекурсивно раскрывает все записи `CNAME`. То есть, если у домена есть поле `CNAME`, нужно рекурсивно запускать `dig`, пока не найдётся запись `A` / `AAAA` типа, указывающая реальный IP-адрес. Каждое промежуточное имя записывайте в список. Кроме того, выведите имена авторитетных DNS-серверов, на которых лежит ваш домен. Сохраните результаты в JSON-файл `domain.json` (`domain` – значение переданного аргумента). Формат JSON-файла:

```
{
  "domain": "example.com",
  "records": {
    "A": ["93.180.63.127", …],
    "AAAA": ["2001:db8::1", …],
    "MX": ["mail.example.com", …],
    "NS": ["ns1.example.com", …],
    "CNAME": ["cdn.example.net"]
  },
  "cname_chain": ["example.com", "cdn.example.net", "edge.example.org"],
  "authoritative_nameservers": ["ns1.example.com", "ns2.example.com"],
  "errors": []
}
```

Обратите внимание: `json['records']['NS']` и `json['authoritative_nameservers']` – разные сущности и получаются разными запусками `dig`. Первое – записи типа `NS`, возвращённые текущим DNS-резолвером. Второе – `NS`-записи из секции `"Answer"`, то есть реальные авторитетные серверы, которые отвечают за домен на момент запроса. В `hw3/README.md` опишите, почему они могут различаться, на ваш взгляд.

Задание 4. Разбор ARP-кэша подсети.

Вы можете прочитать ARP-кэш ядра в системе линукс с помощью просмотра файла **/proc/net/arp**. Разберитесь, что это за файл. Напишите программу **hw3/ARP_scanner.py**, которая выводит в JSON-файл **arp_scan.json** пары (IP, MAC)-адресов активных устройств в подсети (см. пример ниже). Подумайте, все ли устройства вашей подсети находятся там? Проверьте, подключив и отключив телефон от WiFi. Опишите наблюдения в **hw3/README.md**. Запустите в командной строке **ping 8.8.8.8**. Снова проверьте, что лежит в JSON-файле. Что произошло и почему? Подумайте, что такого сделал **ping**. Пример JSON-файла:

```
{
  "devices": [
    {"ip": "10.1.0.1", "mac": "aa:bb:cc:dd:ee:ff"},
    {"ip": "10.1.0.10", "mac": "11:22:33:44:55:66"}
  ]
}
```

Задание 5. Определение производителя MAC-адреса

Посмотрите на сайт IEEE (Institute of Electrical and Electronics Engineers) и их **стандарты** выдачи MAC-адресов производителям. Существует несколько списков интерфейсов MAC: MA-L, MA-M и MA-S. Разберитесь и поймите, какой список интерфейсов нужен, если хочется определить производителя интерфейса компьютера, телефона или роутера/свитча (маршрутизатора/коммутатора). Напишите скрипт **hw3/mac_lookup.py**, который принимает на вход файл с MAC-адресами (**--macfile**), а на выход в JSON-файл **macfile.json** (**macfile** – значение переданного названия с MAC-адресами) печатает JSON-список производителей этих адресов (см. пример ниже). Считайте, что в файле **--macfile** лежат MAC-адреса, по одному на строке. Опишите в **hw3/README.md**, какие типы списков IEEE вы видели, и какой выбрали в итоге – для правильной работы вашего скрипта вам нужно будет его скачивать (либо каждый раз в рантайме делать HTTP-запрос по URL этого списка, чтобы получать файл). Пример выходного JSON-файла:

```
{
  "devices": [
    {"mac": "F8:4C:DA:45:3D:E1",
     "vendor": "HUAWEI TECHNOLOGIES CO.,LTD"},
    {"mac": "B8:A3:77:D4:50:1F",
     "vendor": "Cisco Systems, Inc"}
  ]
}
```

Задание 6. Шифрование с разными ключами.

Разберитесь с мануалом пакета **openssl**. Из всего пакета вам понадобятся: **openssl rand**, **openssl enc**, **openssl genrsa**, **openssl rsa**, **openssl dgst**. В репозитории **github.com/dkhlebn/FBB-orchestration** в папке с домашними заданиями лежат несколько файлов:

- **HW3_ciphertext.bin** (шифротекст сообщения, зашифрованное с помощью шифра AES-256-CBC);
- **HW3_AES_key1.bin** и **HW3_AES_key2.bin** (симметричные ключи шифрования AES-256-CBC);

- **`HW3_Signature.bin`** (RSA-подпись ключа AES);
- **`HW3_public.pem`** (публичный ключ RSA, позволяющий подтвердить ключ).

Выгрузите себе эти файлы. С помощью **`openssl`** проверьте, каким из ключей AES зашифровано сообщение (примечание: брутфорсная расшифровка всеми ключами НЕ считается за решение, подумайте, как нужно правильно это понять). Расшифруйте сообщение. С помощью **`openssl`** сгенерируйте пару из публичного и приватного ключей RSA. Закодируйте сообщение вторым симметричным ключом (не тем, которым было зашифровано моё сообщение), затем подпишите AES-ключ, который вы использовали для шифрования, своим приватным RSA-ключом. В репозитории расположите файлы **`hw3/reply_ciphertext.bin`**, **`hw3/reply_signature.bin`** и **`hw3/public.pem`**. Опишите ваши действия и выводы по шифрованию, дешифрованию и определению правильного ключа в **`hw3/README.md`**, там же можете описать и ваши ощущения от домашки.